17: Neural Networks II

Some material taken from:
https://github.com/trekhleb/homemade-machine-learning/tree/master/homemade/neural_network
http://cs231n.github.io/neural-networks-1/

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

The Zoom / Canvas Lecture Plan

- Lecture using Zoom:
  - https://iu.zoom.us/j/362983973
  - Password: ENGR315

- Also available on Canvas
  - Next day

Office Hours

- Ethan is hosting virtual office hours:

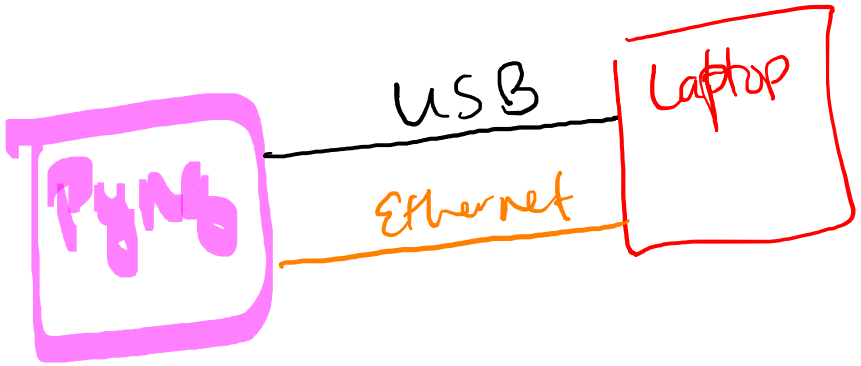
- Wed 5:30 PM - 8:00 PM, Th 2:30 PM - 4:00 PM
- https://iu.zoom.us/j/770949556

Remote Development

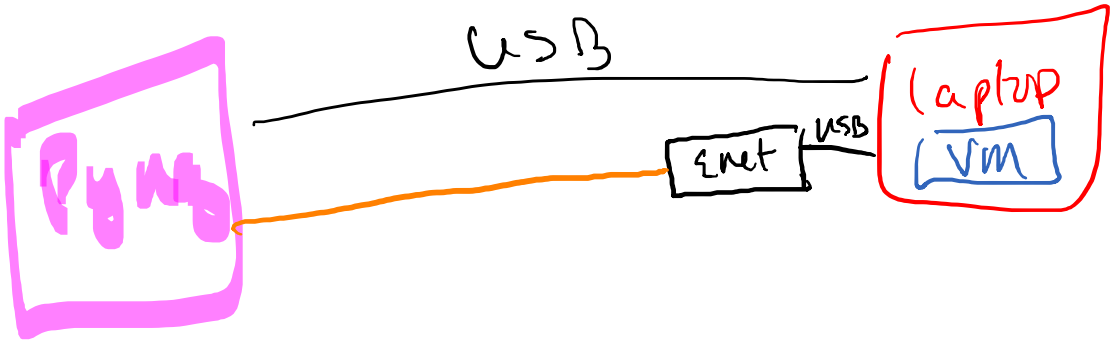
- RED Setup: Allows Vivado on Silo through a web browser

- Pynq Network Setup: Help you get your Pynq networked at home.
  - Direct connection to your laptop
  - Indirect connection through your router

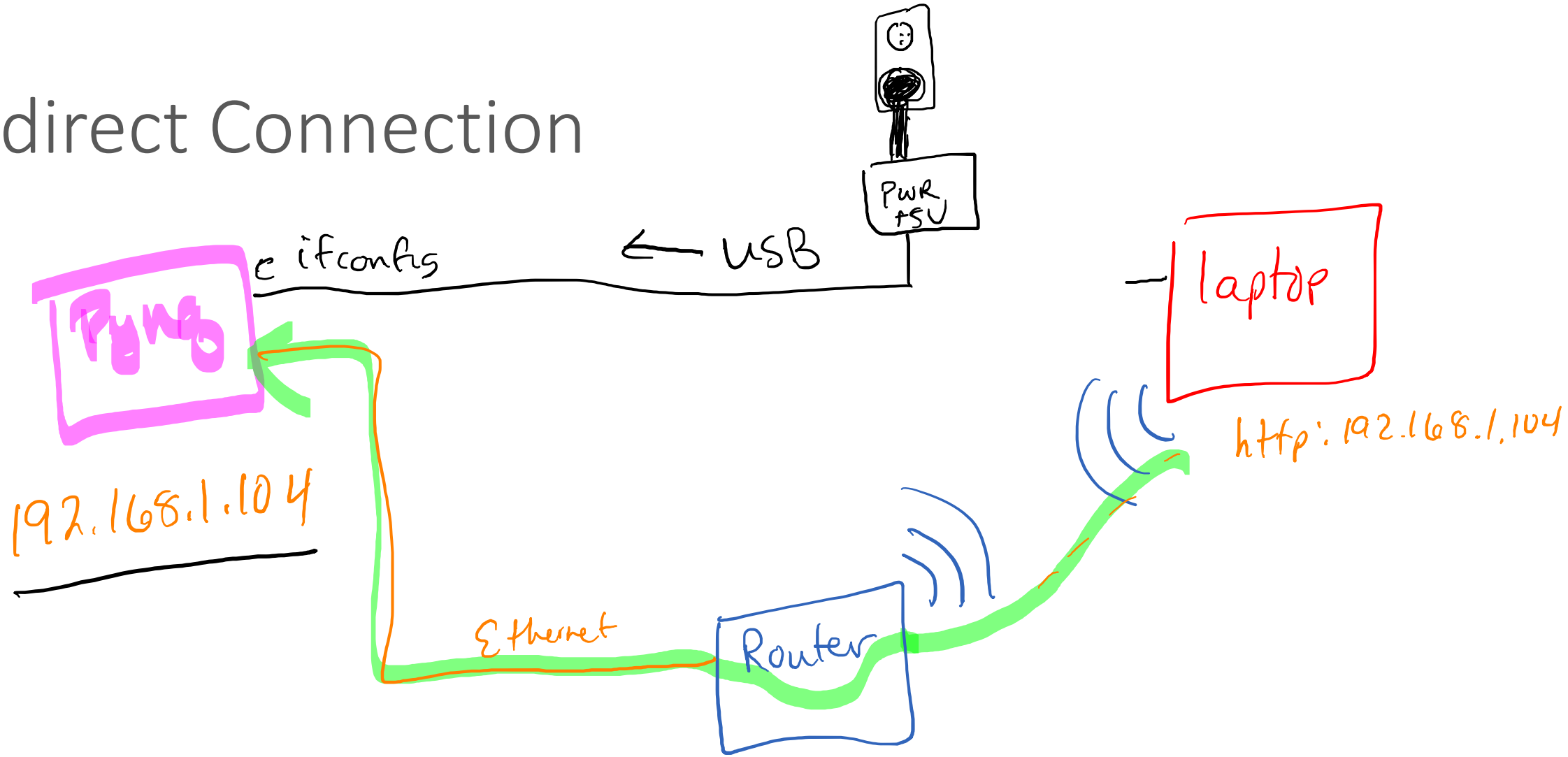
Direct Connection



was setup in 4111



Indirect Connection



- This will change the "IP Address" used in P4.

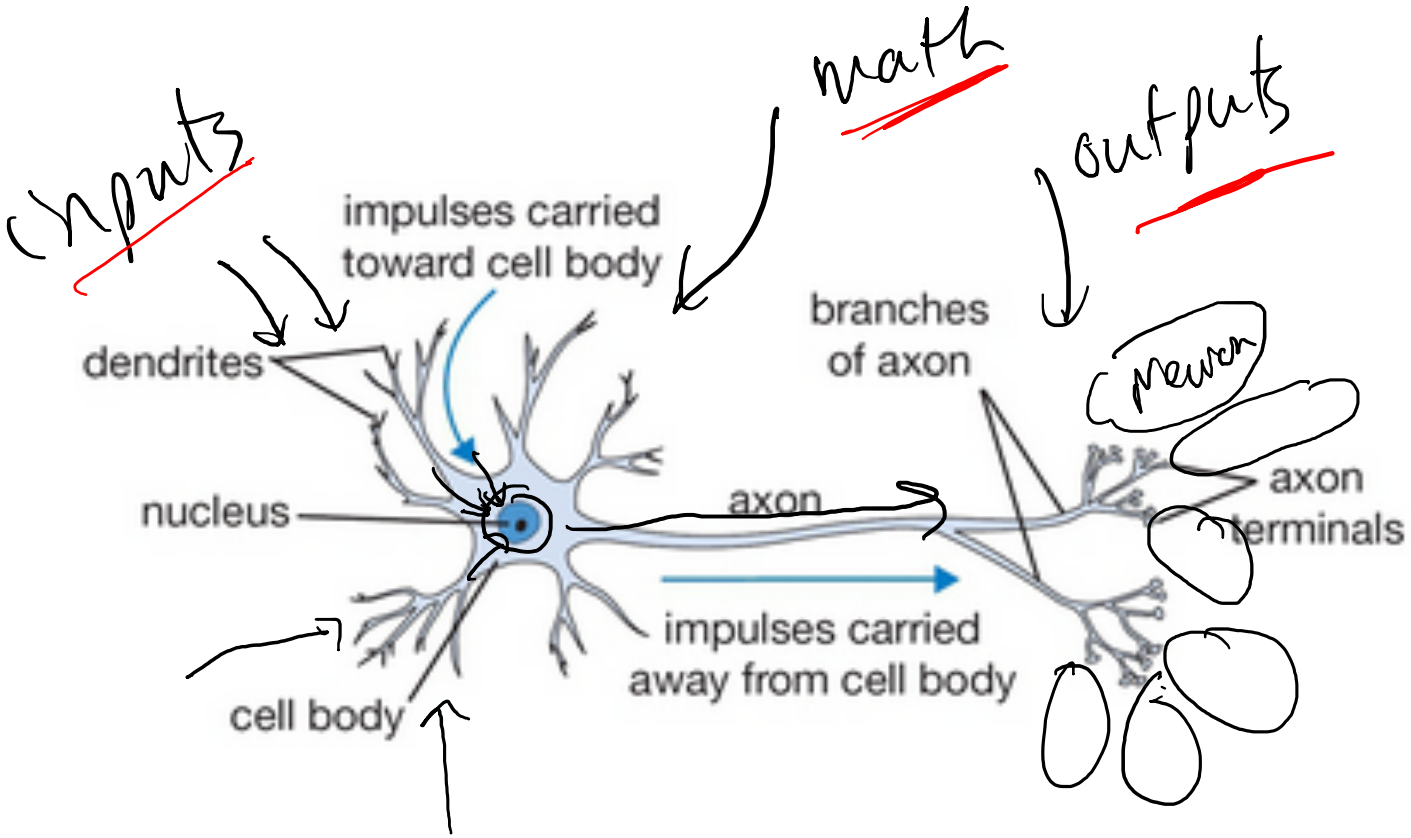
Internet Protocol

Project 4

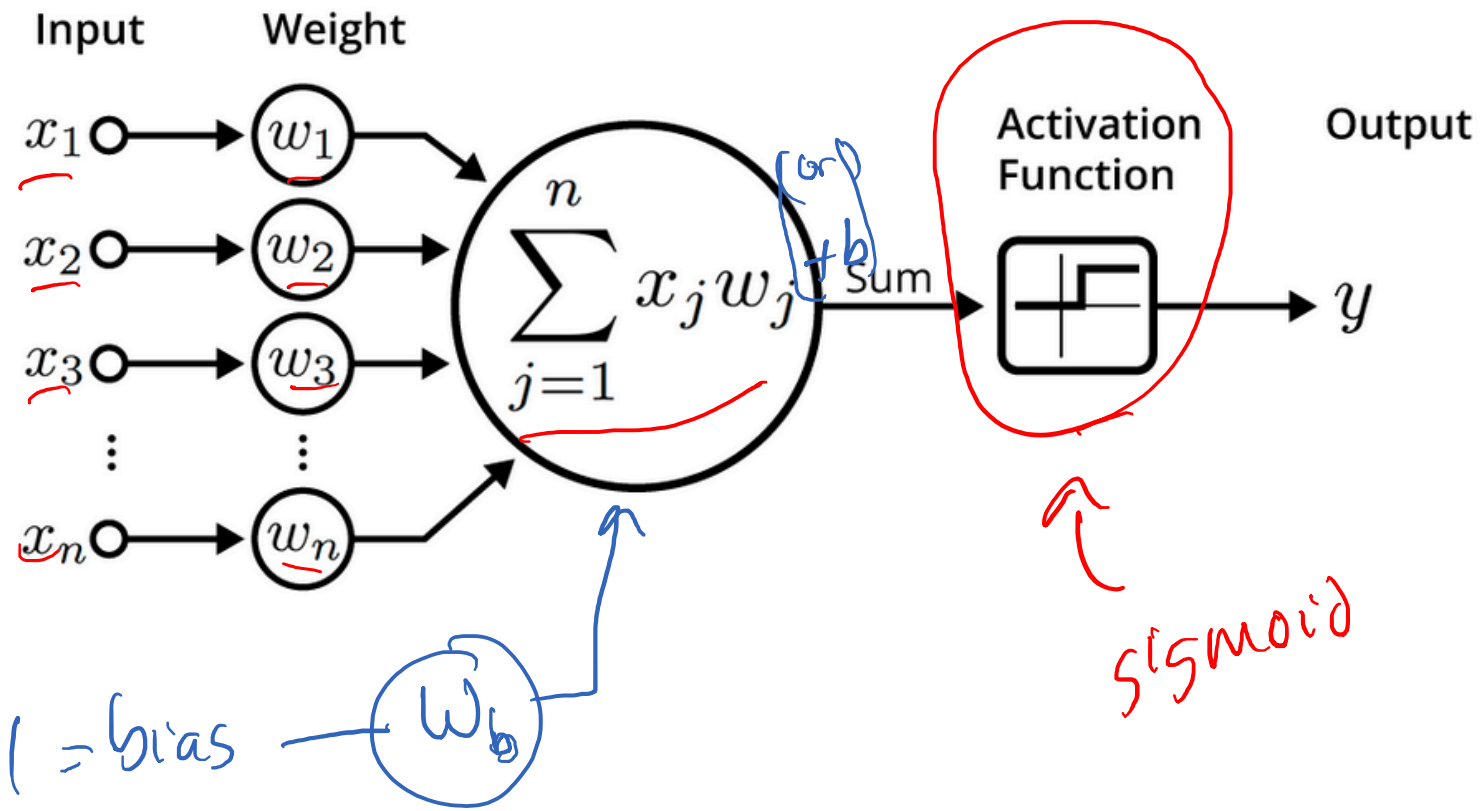
- Remote setup will take some work.
- Pushed back a week.

- Due: April 10th

Last Time: The Biological Neuron



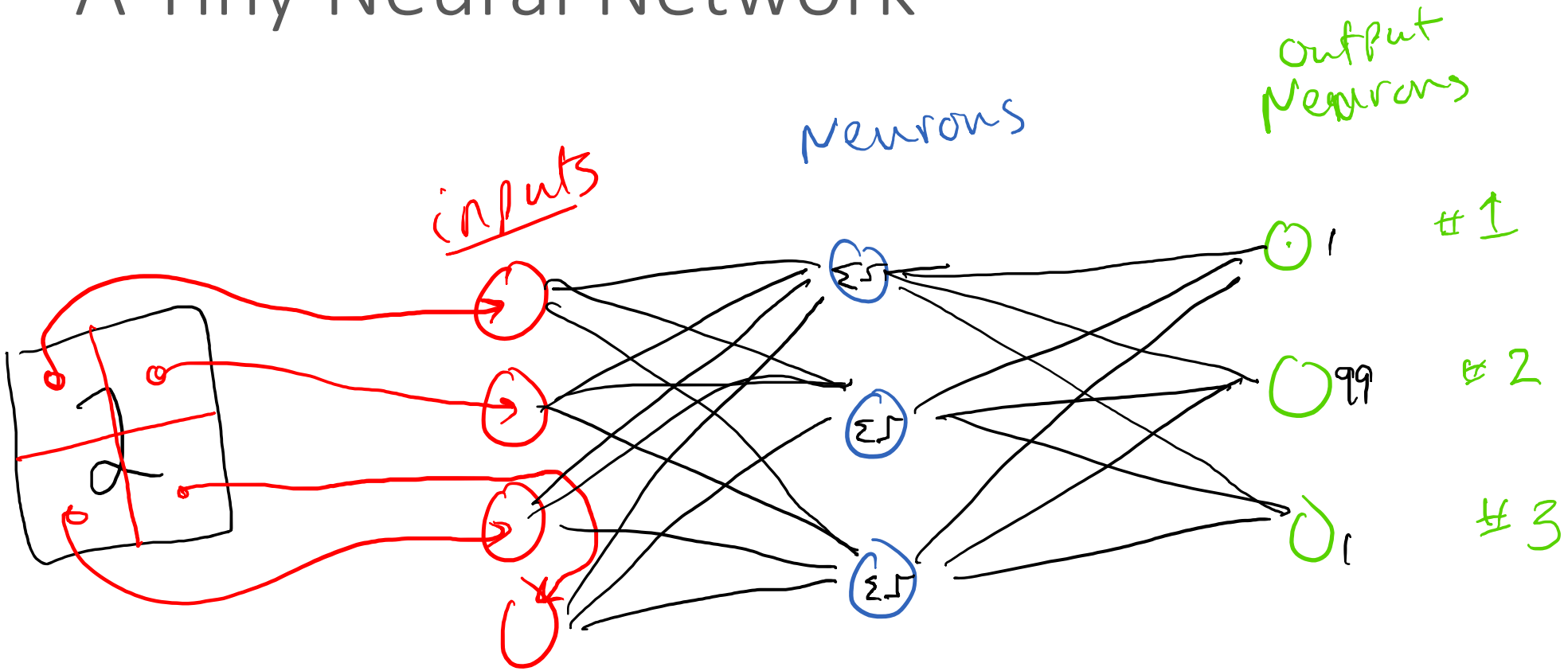
The “Artificial” Neuron

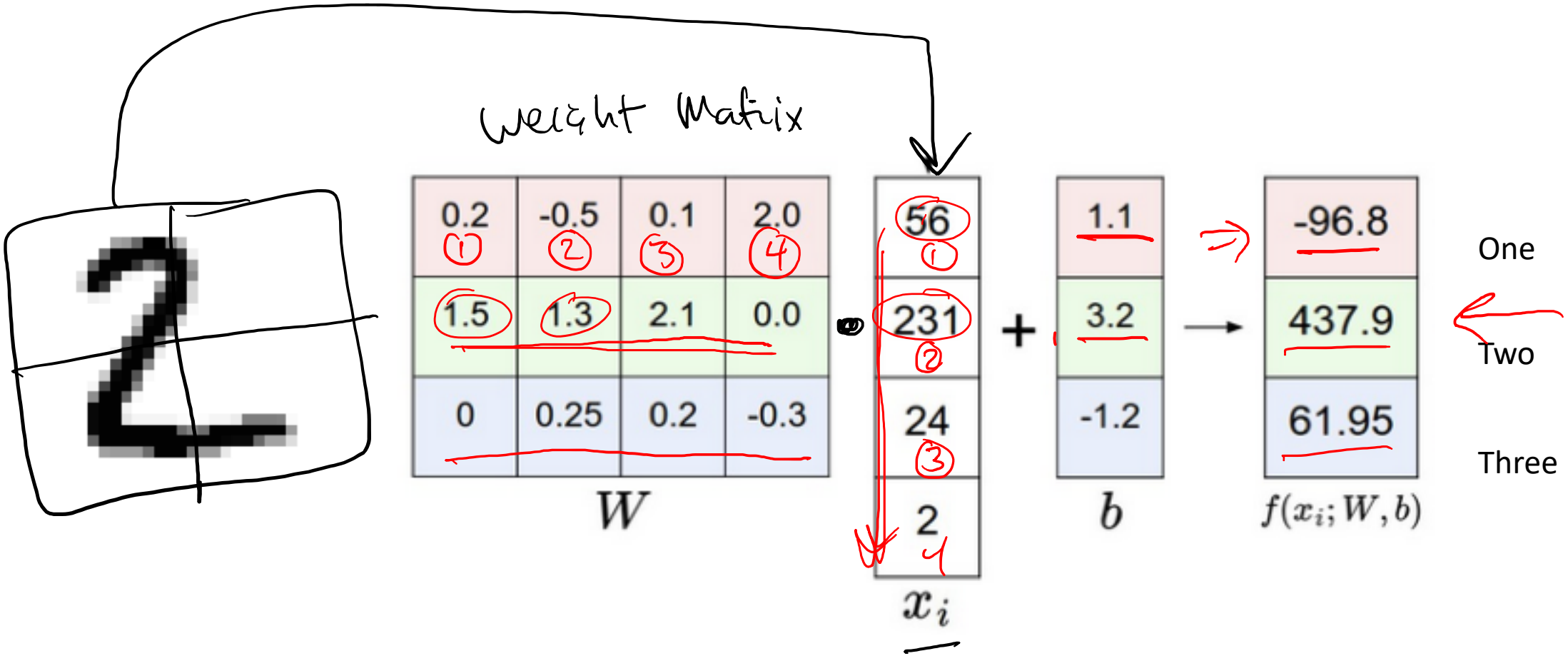


The Python Neuron

```
class Neuron(object):
  # ...
  def forward(self, inputs):
    """ assume inputs and weights are 1-D numpy arrays and bias is a number """
    cell_body_sum = np.sum(inputs * self.weights) + self.bias
    firing_rate = 1.0 / (1.0 + math.exp(-cell_body_sum)) # sigmoid activation function
    return firing_rate
```

A Tiny Neural Network

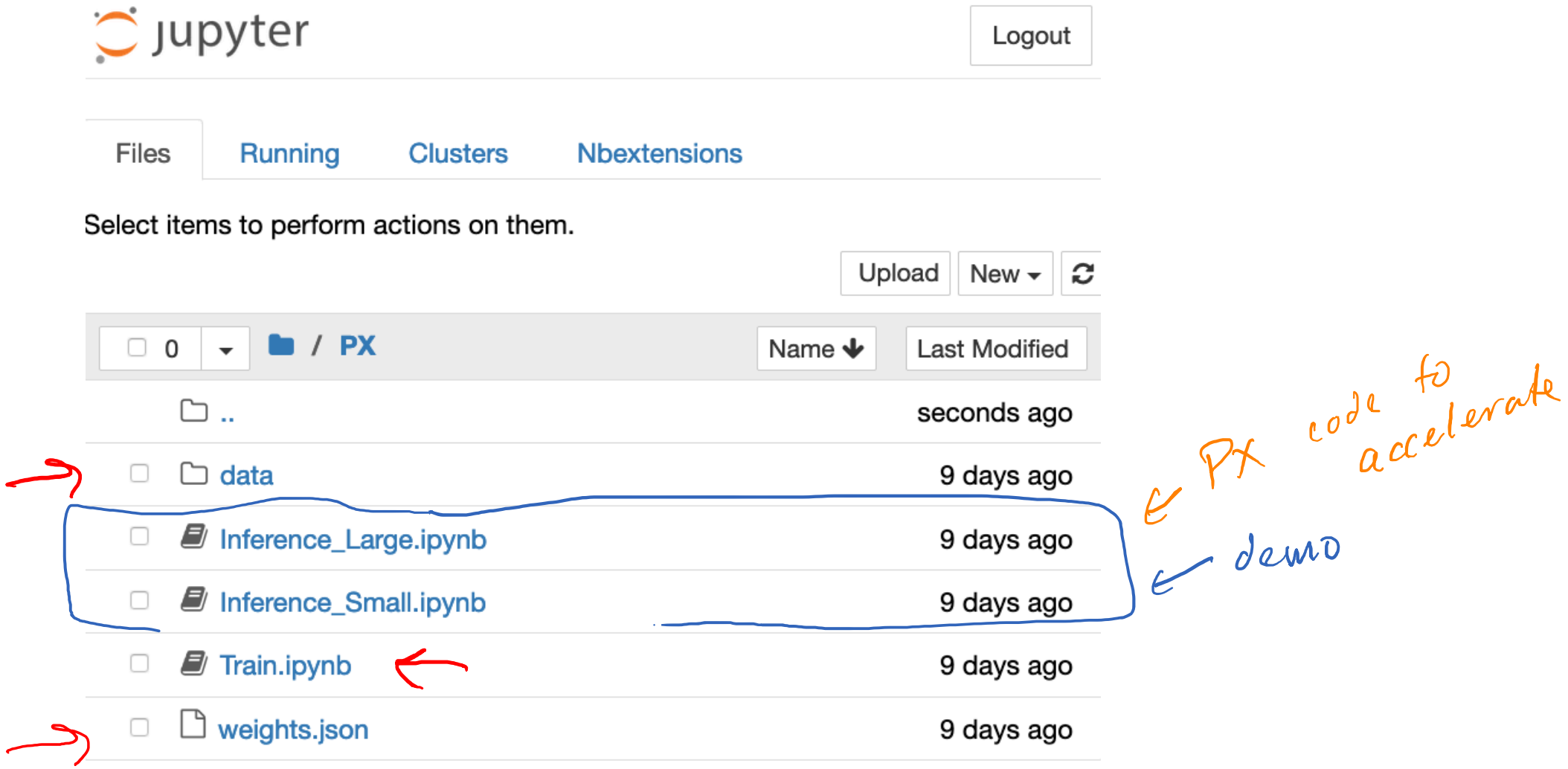




$0.2 \cdot 56 + -0.5 \cdot 231 +$
$0.1 \cdot 24 + 2 \cdot 0.2 =$

input vector

Project X Neural Network Code



What Number is this?

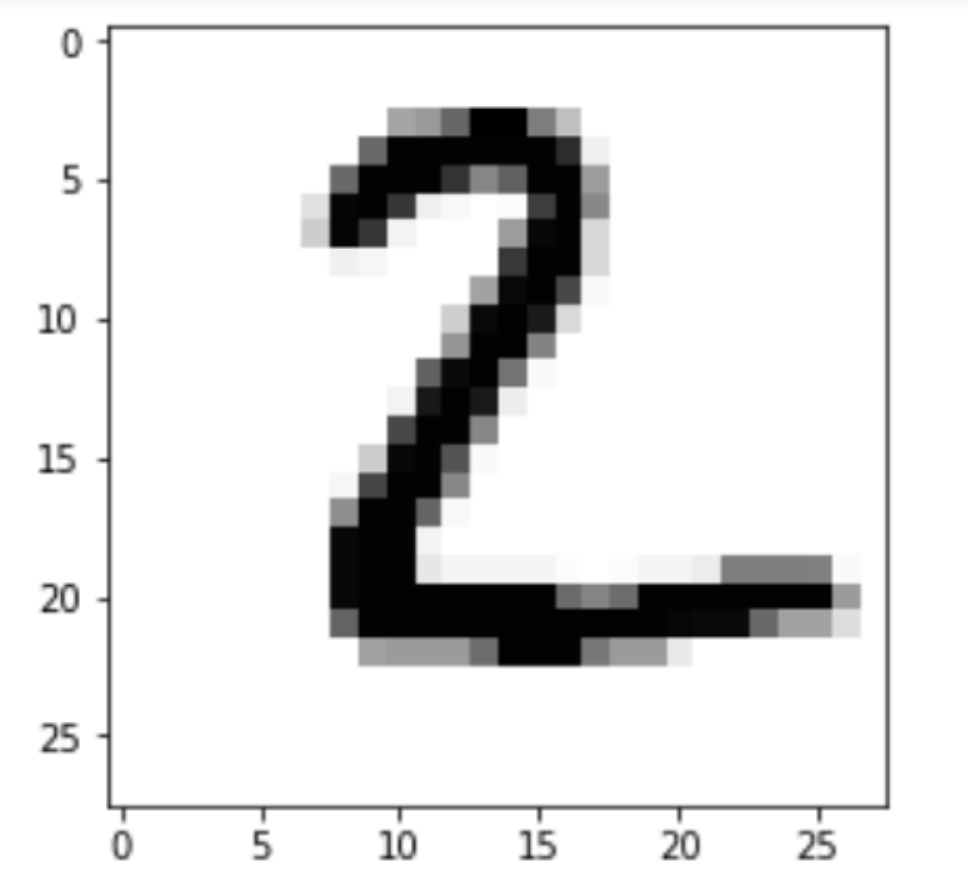
Raw data, in hex

```
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 74 7d ab ff ff 96 5d 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 a9 fd fd fd fd fd fd da 1e 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 a9 fd fd fd d5 8e b0 fd fd 7a 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 34 fa fd d2 20 0c 00 06 ce fd 8c 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 4d fb d2 19 00 00 00 7a f8 fd 41 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 1f 12 00 00 00 00 d1 fd fd 41 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 75 f7 fd c6 0a 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 4c f7 fd e7 3f 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 80 fd fd 90 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 b0 f6 fd 9f 0c 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 19 ea fd e9 23 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 c6 fd fd 8d 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 4e f8 fd bd 0c 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 13 c8 fd fd 8d 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 86 fd fd ad 0c 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 f8 fd fd 19 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 f8 fd fd 2b 14 14 14 14 05 00 05 14 14 25 96 96 96 93 0a 00
00 00 00 00 00 00 00 00 f8 fd fd fd fd fd fd fd a8 8f a6 fd fd fd fd fd fd fd 7b 00
00 00 00 00 00 00 00 00 ae fd fd fd fd fd fd fd fd fd fd fd f9 f7 f7 a9 75 75 39 00
00 00 00 00 00 00 00 00 00 76 7b 7b 7b a6 fd fd fd 9b 7b 7b 29 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

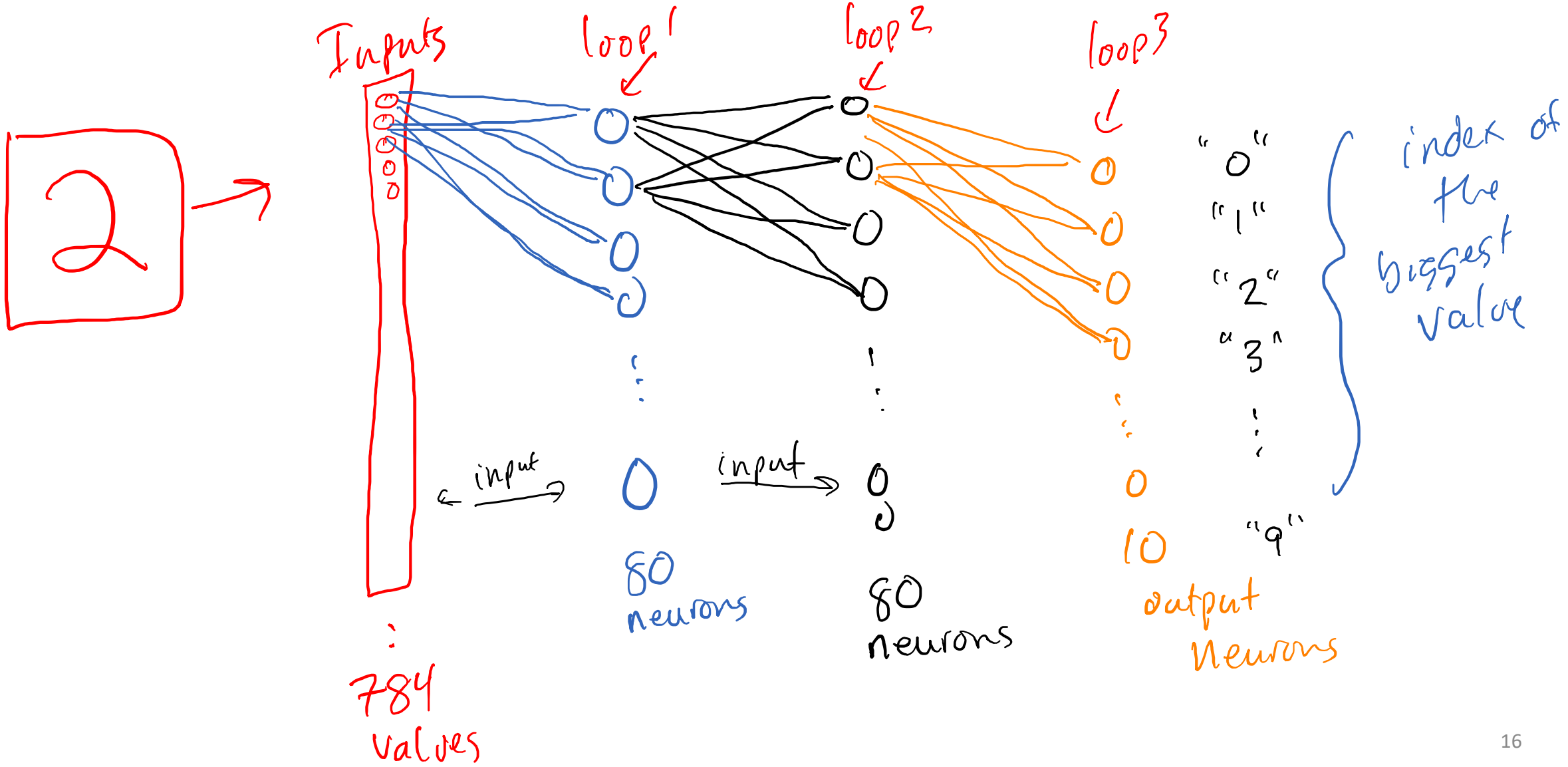
784 pixels

(hex values)

Slightly processed image



Here's the Neural Network model:



The weights are pre-trained in `weights.json`


```
{"structure": [784, 80, 80, 10], "bias": null, "weights":
[[[-0.024668637931634107, 0.0314239133210756, 0.009366978147891974,
0.009890181622188549, 0.03240679908952751, 0.017894864535756828,
-0.014007122491290797, -0.026534907286190245, -0.03592213557819232,
```

...

```
-0.7344105711543865, -0.8216994857556569, 0.30252443812549223,
-0.13211737075290433, 0.2625631280080709, 0.27582328697698494,
0.31103041013057386, -0.07156328881353063, 0.3502124602380533,
-0.5627242185390925, 0.04658157694614686, -0.0010474188856981663,
0.16438392535230725, 0.06739981829880841]]]}
```

Here's the main 'run' loop:

picture

```
def run(self, input_vector):
    # input_vector can be tuple, list or ndarray

    no_of_layers = len(self.structure)
    if self.bias:
        # adding bias node to the end of the inpuy_vector
        input_vector = np.concatenate( (input_vector, [self.bias]) )
    in_vector = np.array(input_vector, ndmin=2).T

    layer_index = 1
    # The input vectors to the various layers
    while layer_index < no_of_layers:
        x = np.dot(self.weights_matrices[layer_index-1],
                   in_vector)
        out_vector = activation_function(x)

        # input vector for next layer
        in_vector = out_vector
        if self.bias:
            in_vector = np.concatenate( (in_vector,
                                         [[self.bias]]) )

        layer_index += 1

    return out_vector.argmax()
```

first 80 neurons

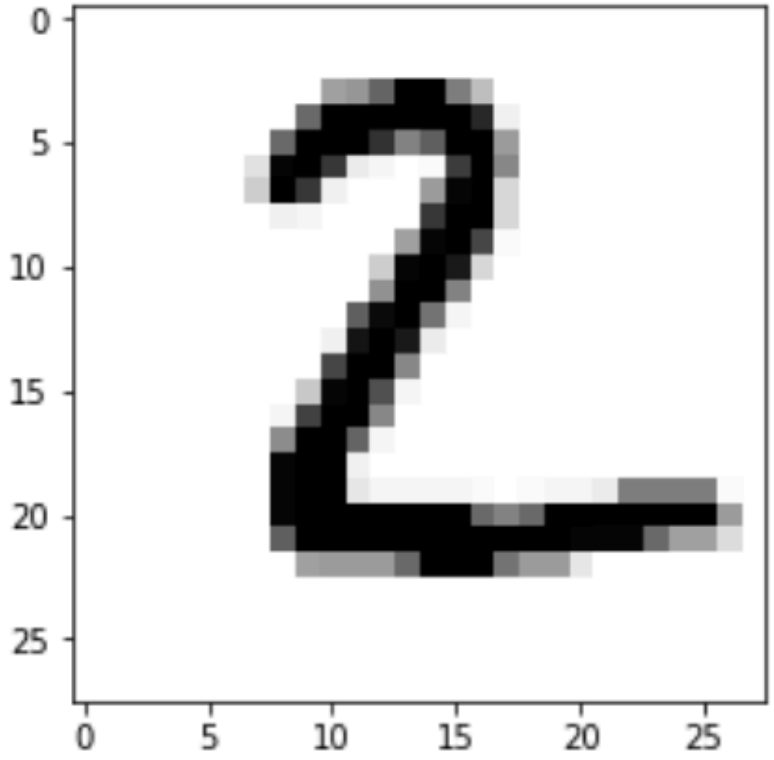
index of biggest value

Let’s create a NN

```
ANN = NeuralNetwork(network_structure=[image_pixels, 80, 80, 10],
                                learning_rate=0.01,
                                bias=None)
ANN.load_weights('weights.json')
```

Let’s see how it does:

```
================================
Index: 0
Image:
```



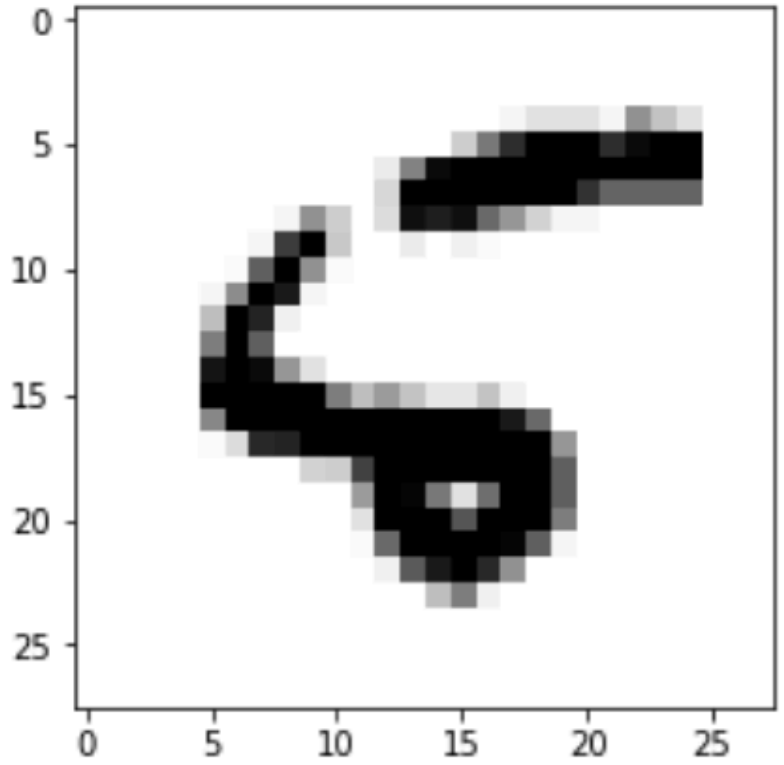

```
ML Classification Result: 2
Real Value: 2
Correct Result: True
================================
```

It’s not always perfect.

5

5

```
================================
Index: 7
Image:
```




```
ML Classification Result: 6
Real Value: 5
Correct Result: False
================================
```

Here's an '`evaluate`' helper function

```
def evaluate(self, data, labels):
    corrects, wrongs = 0, 0
    for i in range(len(data)):
        res = self.run(data[i])
        if res == labels[i]:
            corrects += 1
        else:
            wrongs += 1
    return corrects, wrongs
```

Let’s try 1000 samples

```
corrects, wrongs = ANN.evaluate(test_imgs, test_labels)
print ("Total Correct:" + str(corrects))
print ("Total Incorrect: " + str(wrongs))
print("Overall Accruracy: " + str(corrects / ( corrects + wrongs)))
print("Overall Accruracy (%): " + str(
    int( 1000* corrects / ( corrects + wrongs)) / 10) + "%")
```

```
Total Correct:944
Total Incorrect: 56
Overall Accruracy: 0.944
Overall Accruracy (%): 94.4%
```

How long did that take?

```
start = timeit.default_timer()

corrects, wrongs = ANN.evaluate(test_imgs, test_labels)

stop = timeit.default_timer()
print('Run Time: ' + str(stop - start) + ' Seconds')
```

```
Run Time: 1.7062111091800034 Seconds
```

Where is that time going?

strings

```
cProfile.run('ANN.evaluate(test_imgs, test_labels)')
```

```
         10005 function calls in 1.787 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
     1000    0.137    0.000    1.719    0.002 <ipython-input-7-fb2fdc6122b7>:133(run)
        1    0.068    0.068    1.787    1.787 <ipython-input-7-fb2fdc6122b7>:161(evaluate)
     3000    0.534    0.000    0.534    0.000 <ipython-input-7-fb2fdc6122b7>:4(sigmoid)
        1    0.000    0.000    1.787    1.787 <string>:1(<module>)
        1    0.000    0.000    1.787    1.787 {built-in method builtins.exec}
     1001    0.003    0.000    0.003    0.000 {built-in method builtins.len}
     1000    0.121    0.000    0.121    0.000 {built-in method numpy.core.multiarray.array}
     3000    0.901    0.000    0.901    0.000 {built-in method numpy.core.multiarray.dot}
     1000    0.023    0.000    0.023    0.000 {method 'argmax' of 'numpy.ndarray' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' object
s}
```

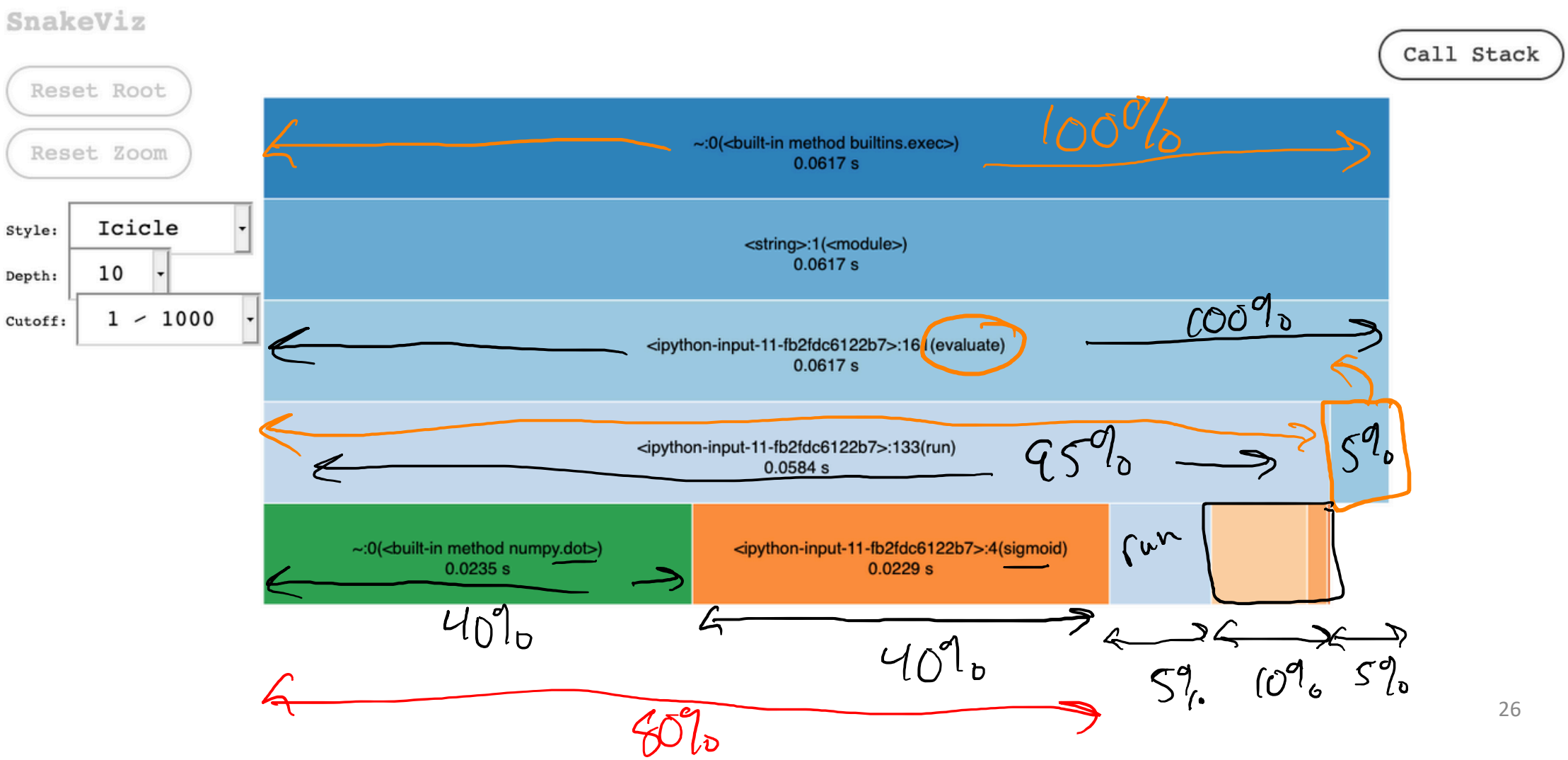
3rd

1st level

2nd level

array allocation

Here's that visualized:



Want to see the source?

Pure-python training code is there also

Project X Assignment:

0 / PX	Name	Last Modified
..		seconds ago
data		9 days ago
Inference_Large.ipynb		9 days ago
Inference_Small.ipynb	Running	3 hours ago
Train.ipynb	Running	9 days ago
test.cprof		3 hours ago
weights.json		9 days ago

==========

This is the part you need to accelerate:

==========

In [ ]:
```
start = timeit.default_timer()

corrects, wrongs = ANN.evaluate(test_imgs, test_labels)

stop = timeit.default_timer()

print ("Total Correct:" + str(corrects))
print ("Total Incorrect: " + str(wrongs))
print("Overall Accruracy: " + str(corrects / ( corrects + wrongs)))
print("Overall Accruracy (%): " + str( int( 1000* corrects / ( corrects + wrongs)) / 10) + "%")
print ()
print('Run Time: ' + str(stop - start) + ' Seconds')
```

For reference, my Pynq takes 17.63 seconds for the above evaluation.

Your mission: Make this run faster, while keeping the 95.3% overall accuracy!

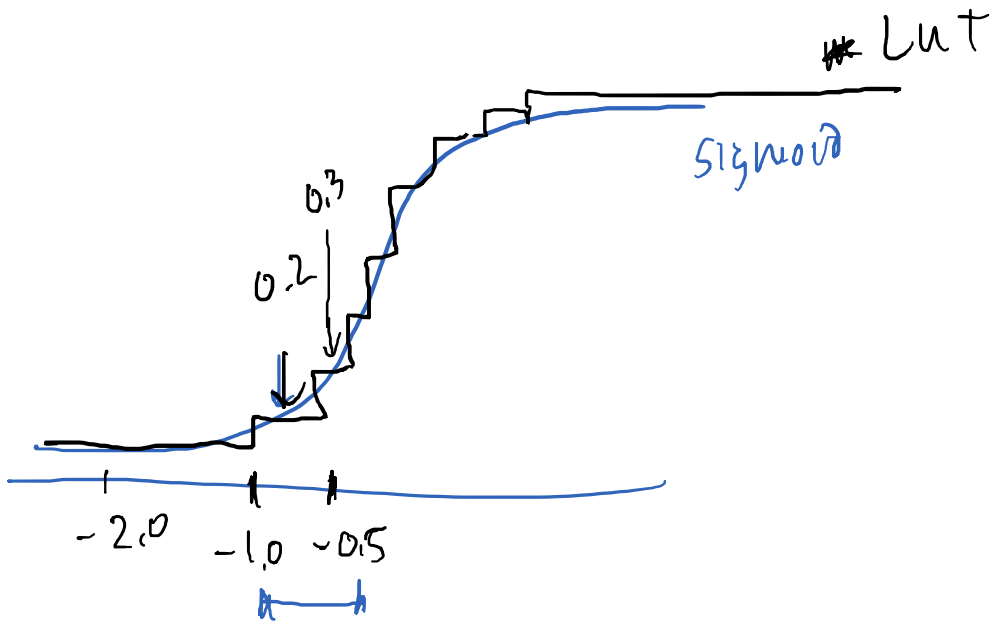
→ faster w/ ↑ accuracy is fine

→ different subset of incorrect, but 95.3% overall

Next Time

- Start to look at how to optimize it.

Unwanted Quantization



17: Neural Networks II

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University